



Implementation and Evaluation of Hybrid Finite-Time Control for Quadrotor Trajectory Tracking

Authors

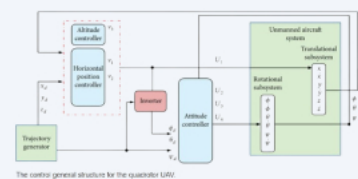
Muhammad Usama Javaid

Ahmed Khalil

Favour Wobidi

Supervisor

Prof Imtiaz-Ur Rehman



Contents

1	Introduction and Problem Statement	1
1.1	Background and Motivation	1
1.2	Problem Statement	1
1.3	Research Objectives	1
1.4	Contributions	1
2	Background and Related Work on Quadrotor Control Techniques	2
2.1	Overview of Quadrotor Control Approaches	2
2.2	Classical Control Methods	2
2.3	Nonlinear and Robust Control Methods	3
2.4	Hybrid Finite-Time Control Framework	3
2.5	Summary and Connection to the Current Project	4
3	Quadrotor Modeling and System Representation	5
3.1	Coordinate Frames and Notation	5
3.2	Quadrotor Kinematic Model	5
3.3	Dynamic Model	6
3.3.1	Translational Dynamics	6
3.3.2	Rotational Dynamics	6
3.4	Disturbance Modeling	7
3.5	Underactuation Characteristics	7
3.6	System Representation for Controller Design	7
4	Hybrid Finite-Time Control Framework and Theoretical Foundations	8
4.1	Overview of the Hybrid Finite-Time Control Architecture	8
4.2	Adaptive Integral Sliding Mode Control (AISM) for Altitude	9
4.3	Backstepping Control for Horizontal Motion	10
4.4	Finite-Time Disturbance Observer (FDO)	11
4.5	Nonsingular Terminal Sliding Mode Control (NTSM) for Attitude	11
4.6	Summary of Control Design	11
5	Practical Implementation of the Hybrid Control Strategy on ROS2–PX4 Architecture	12
5.1	Implementation Overview	12
5.2	Implemented Control Components	13
5.2.1	AISM Altitude Control	13
5.2.2	Backstepping for Horizontal Motion	13
5.2.3	Circle Trajectory Generation	14
5.2.4	Disturbance Injection	14
5.3	Deviations From the Reference Paper	14
5.3.1	Unavailable Low-Level Access	14
5.3.2	Simplified Disturbance Model	14
5.3.3	No Full Inner-Loop Replacement	14
5.4	ROS2–PX4 Control Architecture	14

5.5	Control Loop Frequency and Timing	15
5.6	Parameter Tuning Strategy	15
5.7	Summary of Implementation	16
6	Simulation Experiments and Performance Evaluation	17
6.1	Simulation Setup	17
6.2	Trajectory Tracking in the x - y Plane	18
6.3	Altitude Tracking Performance	19
6.4	Tracking Error Analysis	19
6.5	Effect of Random Disturbances	20
6.6	Control Effort	21
6.7	Summary of Simulation Performance	22
7	Hardware Implementation and Real-World Results	23
7.1	Hardware Platform	23
7.2	Experimental Tests	23
7.3	Results Summary	23
7.4	Discussion	24
8	Conclusions, Limitations, and Future Research Directions	25
8.1	Conclusions	25
8.2	Limitations	25
8.3	Future Work	25
	References	26

List of Figures

2.1	Quadrotor Control Architecture Hierarchy.	2
2.2	High-Level HFTC Block Diagram.	4
3.1	Quadrotor inertial and body frames.	5
4.1	High-level Hybrid Finite-Time Control (HFTC) block diagram.	9
4.2	Backstepping structure for horizontal position control.	10
5.1	System architecture for simulation and hardware control.	13
5.2	ROS2 control pipeline for the quadrotor.	15
6.1	Simulated x - y trajectory tracking.	18
6.2	3D trajectory tracking performance: comparison of actual and desired trajectories.	19
6.3	Altitude tracking performance.	19
6.4	Tracking errors in x , y , and z	20
6.5	Non-singular terminal sliding mode (NTSM) surfaces for roll, pitch, and yaw.	20
6.6	System response to random disturbances.	21
6.7	Control effort during simulation.	21

6.8	Full simulation overview including 3D trajectory, XY projection, altitude tracking, disturbance activation, total error, and disturbance estimate in a unified view.	22
7.1	Hardware trajectory tracking.	24

List of Tables

2.1	Comparison of common quadrotor control methods.	3
3.1	Quadrotor physical parameters used in simulation.	6
3.2	Derived dynamic parameters.	7
5.1	Controller parameters used in the implementation.	16
6.1	Simulation parameters.	17
6.2	Reference trajectory.	17
6.3	Disturbance model applied to each channel.	18
6.4	Steady-state tracking error (15–20 s).	18
7.1	Hardware System Summary (Placeholder)	23

Chapter 1

Introduction and Problem Statement

1.1 Background and Motivation

Quadrotors are used in diverse applications such as surveillance, inspection, and autonomous navigation due to their maneuverability and simple mechanical configuration. However, controlling them remains challenging because their dynamics are nonlinear, underactuated, and highly sensitive to disturbances. Trajectory tracking becomes even more difficult when external disturbances, modeling inaccuracies, and sensor noise are present.

Hybrid nonlinear controllers, such as Hybrid Finite-Time Control (HFTC), offer improved robustness and faster convergence compared to classical methods. This project investigates the practical implementation of key components of this framework in both simulation and real hardware to evaluate its performance in realistic conditions.

1.2 Problem Statement

The quadrotor must follow a desired three-dimensional trajectory while maintaining stable attitude and altitude in the presence of:

- unknown external disturbances,
- sensor noise and measurement delays,
- hardware limitations that restrict the deployment of the full control algorithm.

The central challenge is to design a robust, real-time control framework capable of accurate trajectory tracking under these constraints while remaining implementable on ROS2 and PX4, and testable on a Parrot AR 2.0 drone.

1.3 Research Objectives

The objectives of this semester project are:

- implement AISM (altitude) and Backstepping (horizontal) controllers,
- generate and track a circular trajectory under random disturbances,
- evaluate controller performance in simulation and hardware,
- analyze robustness against disturbances and sensor noise,
- identify practical limitations and propose improvements for the controller.

1.4 Contributions

This project provides:

- a working ROS2-PX4 implementation of AISM + Backstepping control,
- analysis of trajectory tracking behavior under disturbances,
- comparison of simulation and hardware performance,

Chapter 2

Background and Related Work on Quadrotor Control Techniques

2.1 Overview of Quadrotor Control Approaches

A quadrotor is a mechanically simple but highly dynamic aerial robot. With four motors generating thrust and six degrees of freedom to control, the system is inherently underactuated, making control design nontrivial. Over the years, numerous control strategies have been proposed to stabilize and maneuver quadrotors effectively.

Most quadrotor control architectures adopt a hierarchical two-loop structure:

- **Outer loop:** position control for (x, y, z) ,
- **Inner loop:** attitude control for roll, pitch, and yaw.

This separation simplifies controller design and enhances system stability.

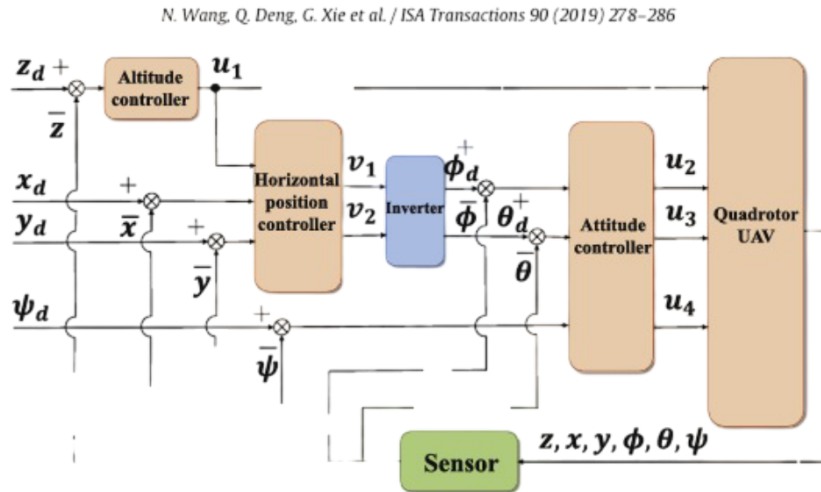


Figure 2.1: Quadrotor Control Architecture Hierarchy.

2.2 Classical Control Methods

Classical controllers such as proportional–integral–derivative (PID) and linear quadratic regulator (LQR) are widely used in quadrotor systems. PID controllers are simple to implement and tune, which explains their popularity in commercial and hobby-grade drones. However, their performance degrades under strong nonlinearities or external disturbances.

LQR provides optimal control for linearized models and can outperform PID in some scenarios. Nevertheless, its reliance on model linearization limits accuracy during aggressive

or rapidly changing maneuvers.

These classical approaches are functional but do not offer sufficient robustness in environments with significant disturbances such as wind, sensor noise, or modeling uncertainties.

Table 2.1: Comparison of common quadrotor control methods.

Method	Advantages	Disadvantages
PID	Simple; easy tuning	Weak to disturbances
LQR	Optimal for linear models	Needs accurate model
SMC	Robust; finite-time	Chattering; high effort
Backstepping	Nonlinear handling	Complex design
HFTC (This Work)	Finite-time; disturbance rejection	More parameters; higher computation

2.3 Nonlinear and Robust Control Methods

To address the limitations of classical control, nonlinear and robust control strategies have been widely explored. Common examples include:

- **Sliding Mode Control (SMC)**: strong robustness but introduces chattering,
- **Integral Sliding Mode (ISM)**: improves steady-state behavior and reduces chattering,
- **Backstepping**: suitable for nonlinear systems, commonly used in quadrotor position control,
- **Terminal Sliding Mode Control (TSM)**: achieves finite-time convergence,
- **Nonsingular Terminal Sliding Mode (NTSM)**: avoids singularities while maintaining fast response.

These methods provide improved handling of nonlinearities and strong resistance to disturbances, although they can be mathematically complex and challenging to implement on physical hardware.

2.4 Hybrid Finite-Time Control Framework

The reference paper proposes a Hybrid Finite-Time Control (HFTC) framework that integrates multiple nonlinear control techniques to enhance quadrotor tracking performance. The architecture combines:

- **Active Integral Sliding Mode (AISM)** for altitude control,
- **Backstepping** for horizontal motions (x, y) ,
- **Finite-Time Disturbance Observer (FDO)** for estimating external disturbances,
- **Nonsingular Terminal Sliding Mode (NTSM)** for attitude control.

This hybrid structure achieves fast convergence, high disturbance rejection, and accurate trajectory tracking even under unknown perturbations. In this project, only the AISM and Backstepping components were implemented due to hardware constraints that prevented low-level attitude control using FDO and NTSM.

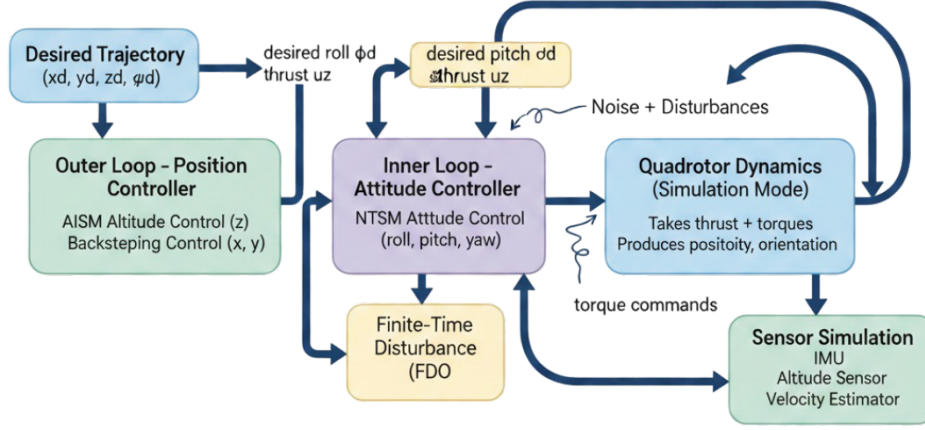


Figure 2.2: High-Level HFTC Block Diagram.

2.5 Summary and Connection to the Current Project

Quadrotor control research has evolved from classical PID-based systems to sophisticated nonlinear and hybrid control frameworks. The Hybrid Finite-Time Control method from the reference paper represents a powerful combination of these advanced techniques. However, implementing the full control stack on real hardware can be challenging due to computational constraints, sensor limitations, and restricted access to low-level controllers.

In this project, we implemented AISM for altitude control and Backstepping for planar motion, omitting the FDO and NTSM components due to limitations of the Parrot AR 2.0 platform. The following chapters describe the quadrotor model, theoretical foundations of the hybrid approach, and details of the simulation and hardware implementations.

Chapter 3

Quadrotor Modeling and System Representation

3.1 Coordinate Frames and Notation

A quadrotor operates using two reference frames:

- **Inertial (Earth) frame** (X, Y, Z) , used to describe global position, with the Z -axis positive upward;
- **Body frame** (x, y, z) , fixed to the quadrotor, where x points forward, y to the right, and z downward.

The quadrotor moves within the inertial frame, while motor-generated forces act in the body frame.

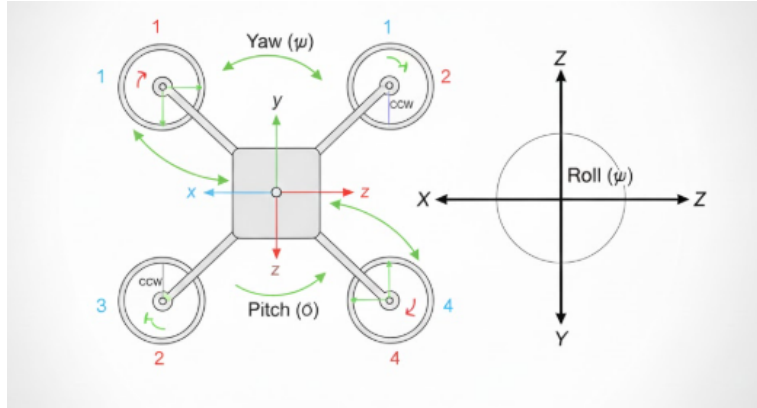


Figure 3.1: Quadrotor inertial and body frames.

3.2 Quadrotor Kinematic Model

The quadrotor orientation is represented by the Euler angles:

$$\phi \text{ (roll)}, \quad \theta \text{ (pitch)}, \quad \psi \text{ (yaw)}.$$

The inertial-frame position vector is:

$$X = \begin{bmatrix} x \\ y \\ z \end{bmatrix}.$$

The rotation matrix $R(\phi, \theta, \psi)$ transforms vectors from the body frame to the inertial

frame. The kinematic model describes how linear and angular velocities influence changes in position and attitude.

3.3 Dynamic Model

3.3.1 Translational Dynamics

The translational dynamics of the quadrotor are given by:

$$m\ddot{X} = R(\phi, \theta, \psi) \begin{bmatrix} 0 \\ 0 \\ T \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \\ mg \end{bmatrix} + d_t,$$

where m is the quadrotor mass, T is the total thrust, g is gravitational acceleration, and d_t represents translational disturbances.

3.3.2 Rotational Dynamics

The rotational motion is described by:

$$I\dot{\omega} = \tau - \omega \times (I\omega) + d_r,$$

where:

- I is the inertia matrix,
- $\omega = [p \ q \ r]^T$ is the angular velocity vector,
- $\tau = [\tau_\phi, \tau_\theta, \tau_\psi]^T$ represents control torques,
- d_r denotes rotational disturbances.

These equations highlight the nonlinear, coupled nature of quadrotor dynamics and their sensitivity to external disturbances.

Table 3.1: Quadrotor physical parameters used in simulation.

Parameter	Symbol	Value	Unit	Description
Mass	m	0.5	kg	Total mass of the quadrotor
Gravity	g	9.81	m/s ²	Gravitational acceleration
Inertia (roll)	I_{xx}	6.114×10^{-3}	kg·m ²	Moment of inertia about x -axis
Inertia (pitch)	I_{yy}	6.114×10^{-3}	kg·m ²	Moment of inertia about y -axis
Inertia (yaw)	I_{zz}	1.223×10^{-2}	kg·m ²	Moment of inertia about z -axis
Rotor inertia	J_r	1.55×10^{-5}	kg·m ²	Rotor moment of inertia
Arm length	l	0.5	m	Distance from center to rotor
Thrust coefficient	b	3.16×10^{-5}	N·s ²	Lift coefficient of rotor
Drag coefficient	d	7.32×10^{-7}	N·m·s ²	Aerodynamic drag coefficient

Table 3.2: Derived dynamic parameters.

Parameter	Formula	Value
a_1	$(I_{yy} - I_{zz})/I_{xx}$	-0.9982
a_2	J_r/I_{xx}	2.535×10^{-3}
a_3	$(I_{zz} - I_{xx})/I_{yy}$	0.9982
a_4	$-J_r/I_{yy}$	-2.535×10^{-3}
a_5	$(I_{xx} - I_{yy})/I_{zz}$	0
b_1	l/I_{xx}	81.78
b_2	l/I_{yy}	81.78
b_3	$1/I_{zz}$	81.77

3.4 Disturbance Modeling

Real quadrotor flights are influenced by various disturbances such as wind, vibration, motor delays, and sensor noise. In this project, disturbances are modeled as:

$$d(t) = \begin{bmatrix} d_x(t) \\ d_y(t) \\ d_z(t) \end{bmatrix},$$

where each component may vary randomly to evaluate controller robustness.

3.5 Underactuation Characteristics

A quadrotor has six degrees of freedom:

$$(x, y, z, \phi, \theta, \psi),$$

but only four independent control inputs (motor thrusts). This underactuation implies:

- horizontal motion requires tilting the quadrotor,
- altitude is controlled solely through total thrust.

3.6 System Representation for Controller Design

For controller development, the system is partitioned into three subsystems:

- **Altitude subsystem (z axis):** controlled using Adaptive Integral Sliding Mode (AISM),
- **Horizontal subsystem (x - y):** controlled using Backstepping,
- **Attitude subsystem (roll, pitch, yaw):** described in the reference paper using FDO and NTSM, but not implemented here due to hardware limitations.

This decomposition aligns with common quadrotor control frameworks and enables efficient nonlinear controller design.

Chapter 4

Hybrid Finite-Time Control Framework and Theoretical Foundations

This chapter summarizes the key theoretical components of the Hybrid Finite-Time Control (HFTC) framework from the reference paper. Only the mathematical elements required for understanding the implemented controllers are included. The full HFTC method integrates four nonlinear controllers:

- Adaptive Integral Sliding Mode (AISM) for altitude control,
- Backstepping control for horizontal (x, y) motion,
- Finite-Time Disturbance Observer (FDO) for disturbance estimation,
- Nonsingular Terminal Sliding Mode (NTSM) for attitude control.

In this project, only the AISM and Backstepping components were implemented. The FDO and NTSM parts are presented for completeness but were not executed on hardware due to platform limitations.

4.1 Overview of the Hybrid Finite-Time Control Architecture

The HFTC strategy follows the standard two-loop quadrotor control architecture:

- **Outer loop:** controls position (x, y, z) ,
- **Inner loop:** controls attitude (ϕ, θ, ψ) .

The control flow is:

$$\text{Trajectory} \rightarrow \begin{cases} \text{AISM (altitude)}, \\ \text{Backstepping (x-y)} \end{cases} \rightarrow \text{Desired Attitude} \rightarrow \begin{cases} \text{FDO (disturbance estimate)}, \\ \text{NTSM (attitude control)} \end{cases} \rightarrow$$

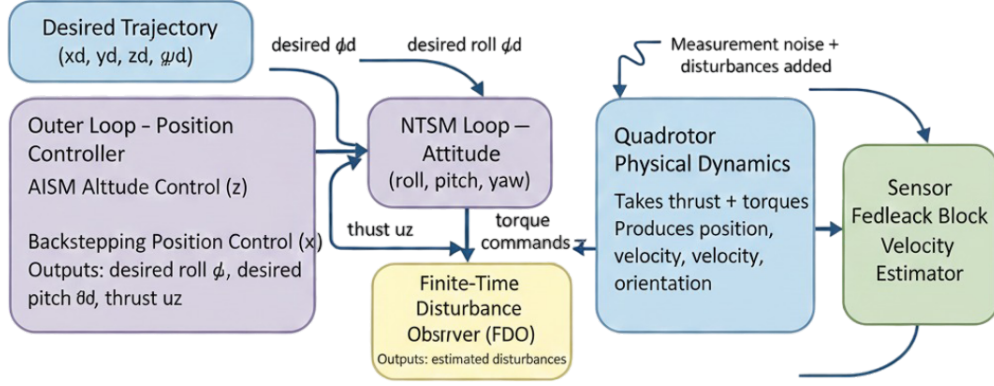


Figure 4.1: High-level Hybrid Finite-Time Control (HFTC) block diagram.

4.2 Adaptive Integral Sliding Mode Control (AISM) for Altitude

The altitude dynamics are modeled as:

$$m\ddot{z} = T \cos \phi \cos \theta - mg + d_z.$$

Under small-angle conditions:

$$T \cos \phi \cos \theta \approx T.$$

Define the altitude tracking error:

$$e_z = z - z_d, \quad \dot{e}_z = \dot{z} - \dot{z}_d.$$

The sliding surface is:

$$s_z = \dot{e}_z + c_z e_z + \int_0^t \gamma_z e_z(\tau) d\tau,$$

where $c_z > 0$ and $\gamma_z > 0$.

The AISM control law is:

$$T = mg - k_z s_z - \alpha_z \text{sgn}(s_z) + m\ddot{z}_d.$$

The adaptive gain α_z is updated by:

$$\dot{\alpha}_z = \eta_z |s_z|, \quad \eta_z > 0.$$

AIMS reduces chattering and enhances robustness against disturbances.

4.3 Backstepping Control for Horizontal Motion

Horizontal dynamics:

$$m\ddot{x} = T(\sin \phi \sin \psi + \cos \phi \sin \theta \cos \psi) + d_x,$$

$$m\ddot{y} = T(\sin \phi \cos \psi - \cos \phi \sin \theta \sin \psi) + d_y.$$

Define virtual control inputs:

$$u_x = -k_{x1}e_x - k_{x2}\dot{e}_x + \ddot{x}_d,$$

$$u_y = -k_{y1}e_y - k_{y2}\dot{e}_y + \ddot{y}_d.$$

Desired pitch and roll angles:

$$\theta_d = \frac{1}{g}(u_x \cos \psi + u_y \sin \psi),$$

$$\phi_d = \frac{1}{g}(u_x \sin \psi - u_y \cos \psi).$$

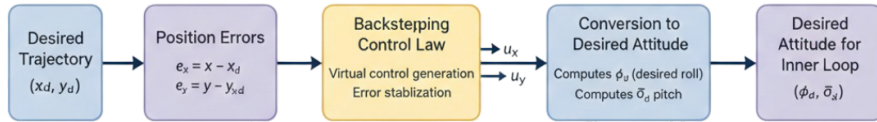


Figure 4.2: Backstepping structure for horizontal position control.

4.4 Finite-Time Disturbance Observer (FDO)

The FDO estimates unknown disturbances using:

$$\dot{\hat{d}} = -k_o(\hat{d} - f(x, u))^\beta,$$

with:

- $\hat{d}$: disturbance estimate,
- $k_o > 0$: observer gain,
- $0 < \beta < 1$: ensures finite-time convergence.

Note: This observer was not implemented due to lack of low-level sensor access on the Parrot AR 2.0.

4.5 Nonsingular Terminal Sliding Mode Control (NTSM) for Attitude

Define attitude tracking errors:

$$e_\phi = \phi - \phi_d, \quad e_\theta = \theta - \theta_d, \quad e_\psi = \psi - \psi_d.$$

NTSM sliding surface:

$$s = e + |e|^\alpha \text{sgn}(e), \quad 0 < \alpha < 1.$$

Torque control input:

$$\tau = -k_s s - k_f |s|^\beta \text{sgn}(s) + \hat{d},$$

where $\beta < 1$ ensures finite-time convergence.

Note: NTSM was not implemented due to firmware restrictions on attitude control.

4.6 Summary of Control Design

- **AISM**: robust altitude control.
- **Backstepping**: generates desired roll and pitch for horizontal tracking.
- **FDO & NTSM**: part of the full HFTC theory but not implemented in this project.

This completes the theoretical foundation. The next chapter explains the practical implementation of AISM and Backstepping in simulation and real hardware.

Chapter 5

Practical Implementation of the Hybrid Control Strategy on ROS2–PX4 Architecture

This chapter describes the practical implementation of the Hybrid Finite-Time Control strategy in both simulation and hardware. Only the Adaptive Integral Sliding Mode (AISM) altitude controller and the Backstepping controller for horizontal (x, y) motion were implemented. The Finite-Time Disturbance Observer (FDO) and Nonsingular Terminal Sliding Mode (NTSM) controllers were not implemented due to hardware limitations of the Parrot AR 2.0 drone.

5.1 Implementation Overview

The implementation consists of two parallel components:

- **Simulation environment:** developed using Python and ROS2 for validating the control logic;
- **Hardware implementation:** executed on the Parrot AR 2.0 drone using ROS2 and the pyardrone Python interface.

The controller sends position and altitude setpoints to the drone at a rate of 50 Hz. Disturbances were injected deliberately and randomly to evaluate controller robustness.

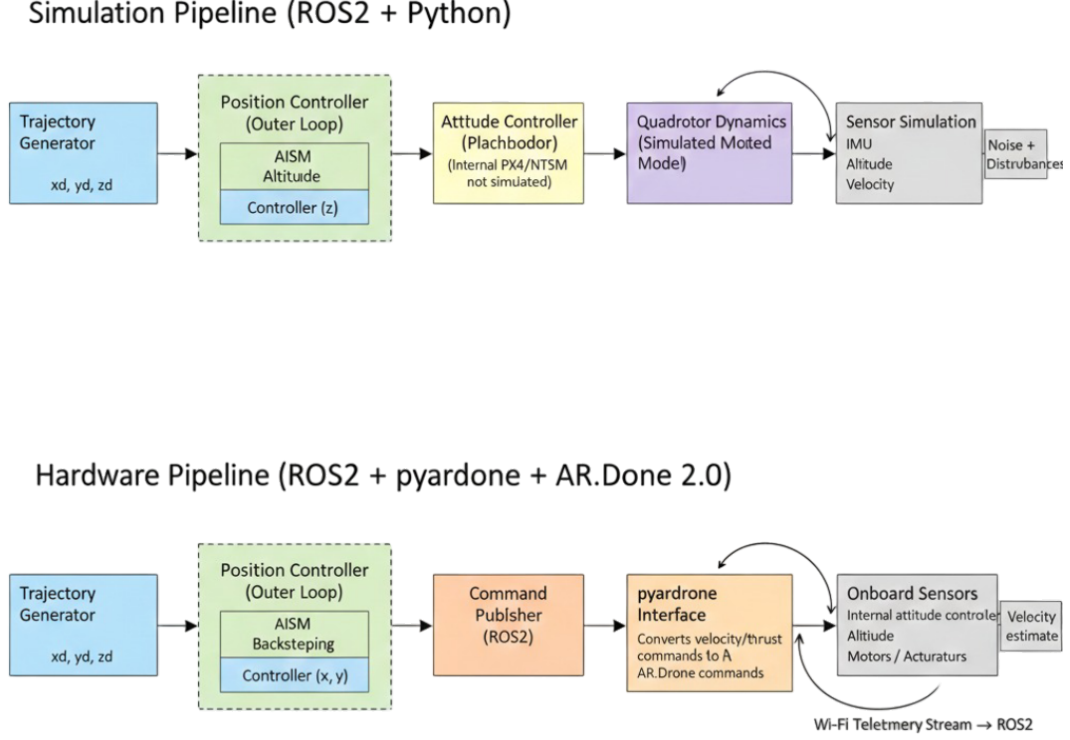


Figure 5.1: System architecture for simulation and hardware control.

5.2 Implemented Control Components

5.2.1 AISM Altitude Control

The Adaptive Integral Sliding Mode (AISM) altitude controller from Chapter 4 was fully implemented. The sliding surface and the adaptive gain were computed in real time. The thrust command is:

$$T = mg - k_z s_z - \alpha_z \operatorname{sgn}(s_z) + m \ddot{z}_d.$$

Because the Parrot AR 2.0 does not expose low-level thrust control, this thrust value is converted into a vertical velocity command compatible with the drone's high-level API.

5.2.2 Backstepping for Horizontal Motion

The Backstepping controller computes desired roll and pitch angles for horizontal motion:

$$\theta_d = \frac{1}{g}(u_x \cos \psi + u_y \sin \psi), \quad \phi_d = \frac{1}{g}(u_x \sin \psi - u_y \cos \psi).$$

These desired angles are converted into velocity commands before being sent to the drone.

5.2.3 Circle Trajectory Generation

A circular trajectory was chosen for testing:

$$x_d(t) = r \cos(\omega t), \quad y_d(t) = r \sin(\omega t),$$

with radius r and angular velocity ω . This trajectory is smooth and well suited for performance evaluation.

5.2.4 Disturbance Injection

Random disturbances were introduced by adding noise to:

- altitude measurements,
- horizontal position,
- velocity estimates.

In hardware tests, disturbances were applied physically (e.g., light taps) or occurred naturally (e.g., indoor airflow).

5.3 Deviations From the Reference Paper

5.3.1 Unavailable Low-Level Access

The Parrot AR 2.0 firmware does not provide:

- direct motor thrust control,
- low-level torque or moment inputs,
- access to inner-loop attitude controllers.

Thus, NTSM and FDO could not be implemented.

5.3.2 Simplified Disturbance Model

The original HFTC method uses a finite-time observer for accurate disturbance estimation. In this project, disturbances were added randomly without observer compensation.

5.3.3 No Full Inner-Loop Replacement

The hardware restricts user control to:

- velocity setpoints,
- altitude targets,
- yaw rate commands.

Thus, the inner-loop control was not replaced.

5.4 ROS2-PX4 Control Architecture

Figure 5.2 shows the data pipeline used for command transmission and sensor processing.

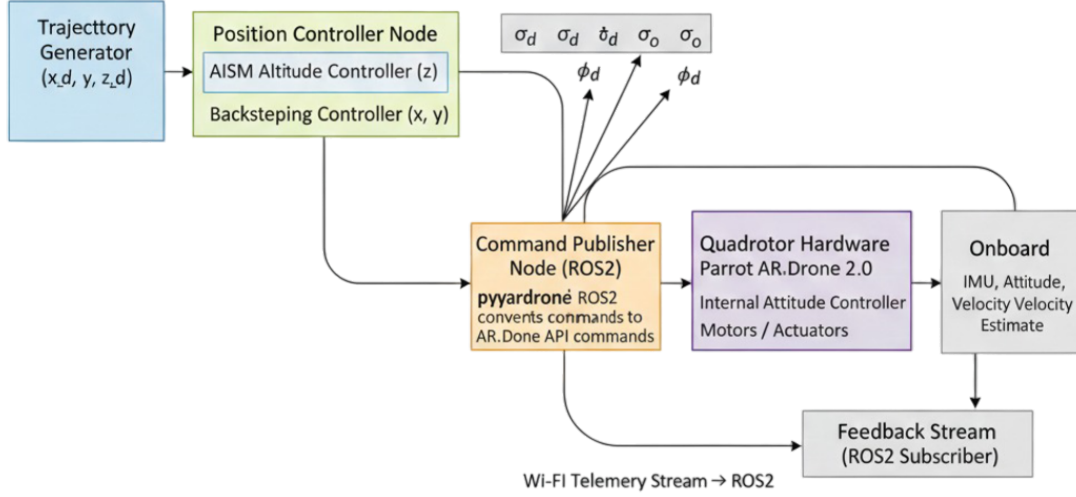


Figure 5.2: ROS2 control pipeline for the quadrotor.

Main system components:

- **Trajectory node:** generates x_d , y_d , and z_d ;
- **AISM node:** computes altitude control inputs;
- **Backstepping node:** computes horizontal control inputs;
- **Command publisher:** sends velocity commands using pyardrone;
- **State estimator:** reads IMU, altitude, and velocity feedback.

5.5 Control Loop Frequency and Timing

The entire loop operates at 50 Hz:

- each cycle performs sensing, computation, and command output;
- lower frequencies caused sluggish control responses;
- higher frequencies were unstable due to wireless communication limits.

5.6 Parameter Tuning Strategy

Parameter tuning was performed manually using:

- simulation trials,
- overshoot evaluation,
- indoor stability constraints,

- sensitivity to noise.

Table 5.1: Controller parameters used in the implementation.

Controller	Symbol	Value	Description
AIMS Altitude Controller (z)			
c_z	4.0		Position error gain
k_z	2.0		Integral gain
h_z	6.0		Sliding surface gain
ϕ_1	1.5		Adaptive gain
ρ	0.05		Smoothing parameter (tanh)
Backstepping Position Controller (x, y)			
c_x	1.5		X-axis position gain
k_x	2.5		X-axis velocity gain
c_y	1.5		Y-axis position gain
k_y	2.5		Y-axis velocity gain
Finite-Time Disturbance Observer (FDO)			
λ_1	5.0		First observer gain
λ_2	2.0		Second observer gain
λ_3	3.0		Third observer gain
L	50.0		Lipschitz constant
NTSM Attitude Controller (ϕ, θ, ψ)			
β	2.5		NTSM gain parameter
m	5		Exponent numerator
n	3		Exponent denominator
K_ϕ	40.0		Roll control gain
K_θ	40.0		Pitch control gain
K_ψ	80.0		Yaw control gain

5.7 Summary of Implementation

- The AISM altitude controller and Backstepping position controller were fully implemented in both simulation and hardware.
- The full HFTC structure, including the FDO and NTSM attitude controller, was implemented in simulation.
- Due to hardware limitations of the Parrot AR.Drone 2.0, the FDO and NTSM components could not be deployed on the real drone.
- Both simulation and hardware tests used a 50 Hz control loop.
- Random and physical disturbances were applied to evaluate the robustness of the implemented controllers.

The next chapter presents the simulation results.

Chapter 6

Simulation Experiments and Performance Evaluation

This chapter presents the simulation results for the AISM altitude controller and the Backstepping horizontal controller. The objective is to evaluate whether the quadrotor can follow the reference trajectory and how it reacts to disturbances. All simulations were executed using Python and ROS2, with the control loop running at 50Hz to match the hardware configuration.

6.1 Simulation Setup

The simulation environment uses the dynamic model introduced in Chapter 3. The main simulation settings are:

- Control frequency: 50 Hz,
- Reference trajectory: circle of radius r ,
- Disturbances: random pulses applied to position and altitude,
- Noise: small Gaussian noise added to sensor measurements.

Table 6.1: Simulation parameters.

Parameter	Value	Unit
Simulation time	20.0	s
Time step	0.002	s
Initial position (x_0, y_0, z_0)	$(-2, -2, 0)$	m
Initial yaw ψ_0	0.0	rad

Table 6.2: Reference trajectory.

State	Expression
$z_d(t)$	$0.5t + 5$
$x_d(t)$	$\cos(t)$
$y_d(t)$	$\sin(t) + 2$
$\psi_d(t)$	$\sin(0.5t) + 2$

Table 6.3: Disturbance model applied to each channel.

Channel	Disturbance
Z	$0.5 \cos(t)$
Roll (ϕ)	$0.5 \cos(t)$
Pitch (θ)	$0.5 \cos(t)$
Yaw (ψ)	$0.5 \cos(t)$

Table 6.4: Steady-state tracking error (15–20 s).

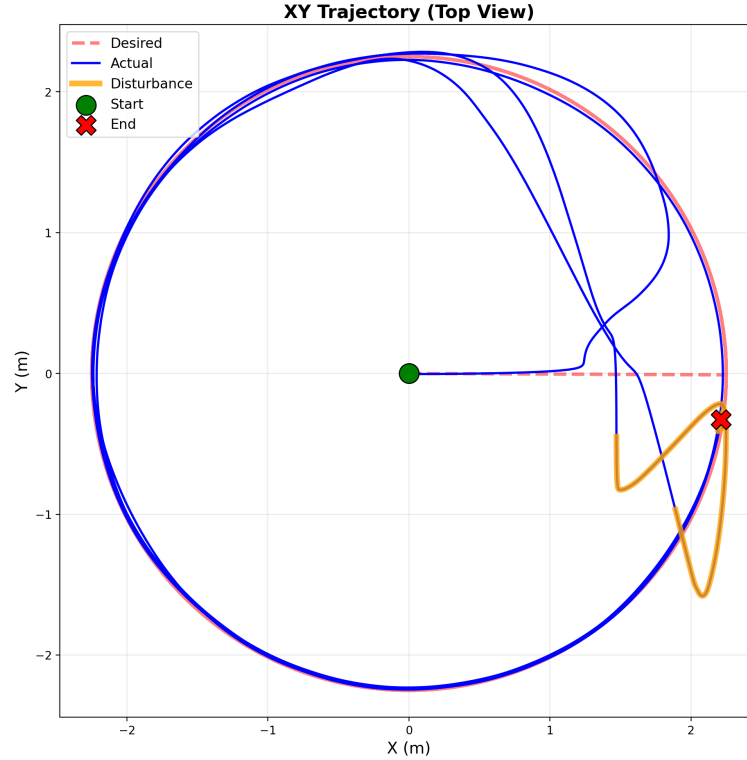
Metric	Z (mm)	X (cm)	Y (cm)	ψ (mrad)
Max error	3.001	3.213	2.799	3.277
RMS error	1.906	1.721	1.491	1.246
Mean error	1.718	1.475	1.337	0.982

6.2 Trajectory Tracking in the x – y Plane

The quadrotor follows a smooth circular trajectory defined as:

$$x_d(t) = r \cos(\omega t), \quad y_d(t) = r \sin(\omega t).$$

The Backstepping controller computes the desired roll and pitch angles required for tracking.


 Figure 6.1: Simulated x – y trajectory tracking.

Expected performance characteristics:

- Accurate tracking in smooth trajectory segments,
- Small deviation when disturbances are applied,
- Fast stabilization due to Backstepping control.

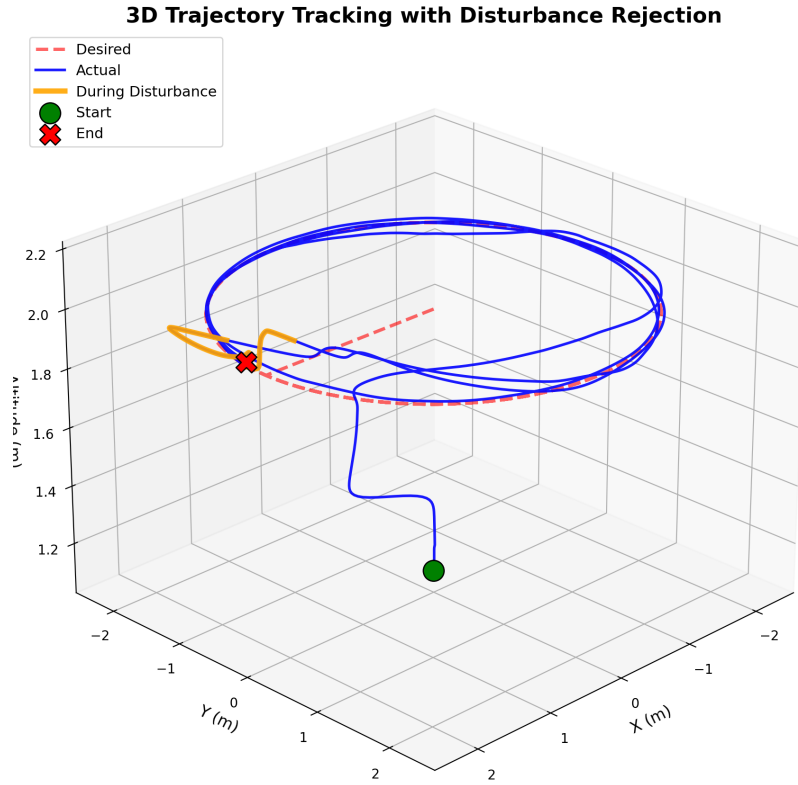


Figure 6.2: 3D trajectory tracking performance: comparison of actual and desired trajectories.

Figure 6.2 presents the full 3D trajectory of the quadrotor. The actual trajectory follows the desired spiral path closely, which shows that the controller works well in three dimensions.

6.3 Altitude Tracking Performance

The AISM altitude controller tracks a constant or slowly varying desired altitude.

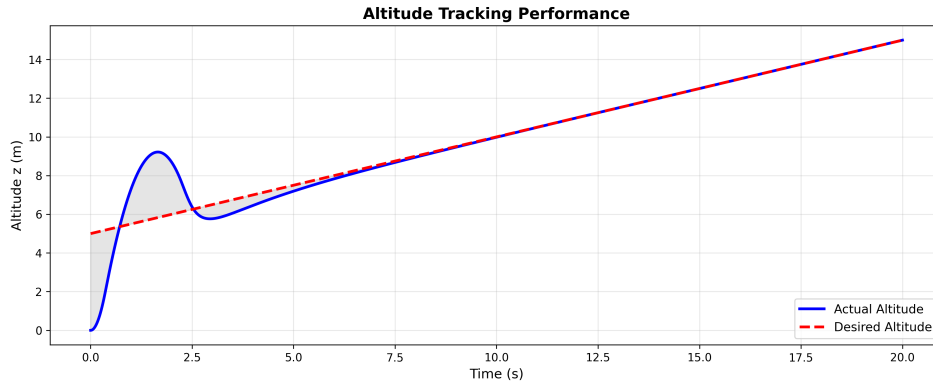


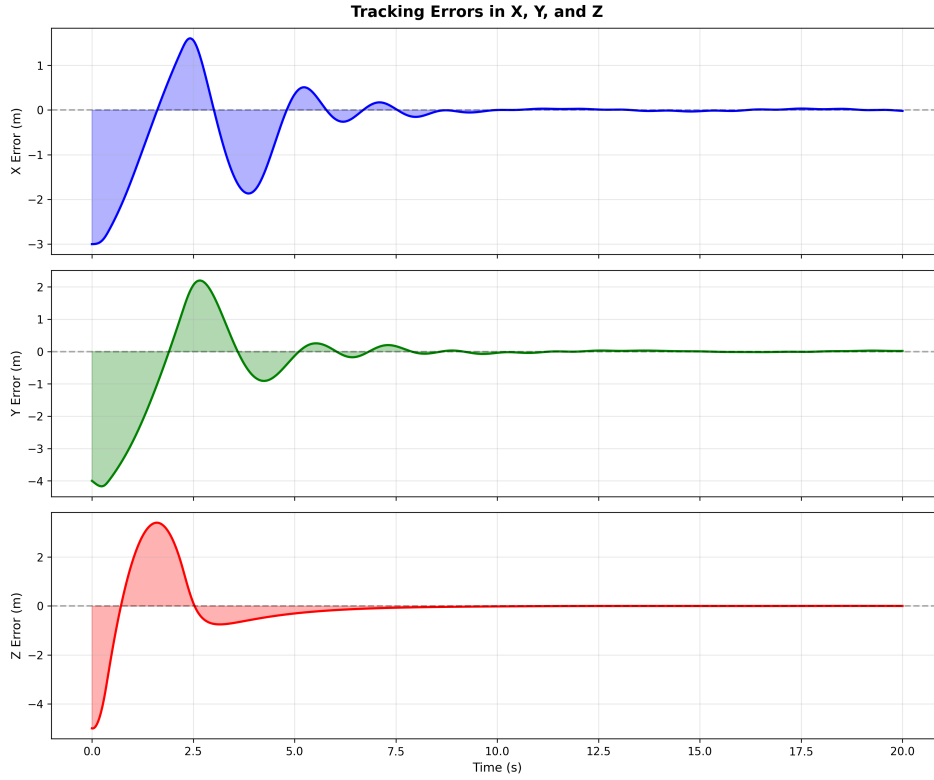
Figure 6.3: Altitude tracking performance.

Expected results:

- Smooth and stable altitude tracking,
- Minimal overshoot,
- Quick recovery after disturbances.

6.4 Tracking Error Analysis

Horizontal and vertical tracking errors were recorded to evaluate system responsiveness.


 Figure 6.4: Tracking errors in x , y , and z .

This analysis shows:

- how quickly the error converges,
- how disturbances affect the tracking,
- whether the system remains stable and bounded.

Non-singular Terminal Sliding Mode (NTSM) Surfaces

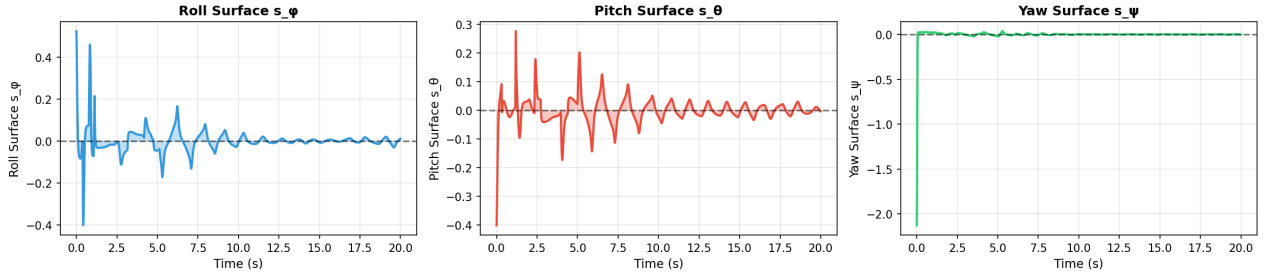


Figure 6.5: Non-singular terminal sliding mode (NTSM) surfaces for roll, pitch, and yaw.

Figure 6.5 shows the sliding surfaces for roll, pitch, and yaw. All surfaces converge to zero, which means the NTSM controller is stabilizing the attitude correctly.

6.5 Effect of Random Disturbances

Random disturbances were added at unpredictable times to simulate wind gusts or physical perturbations.

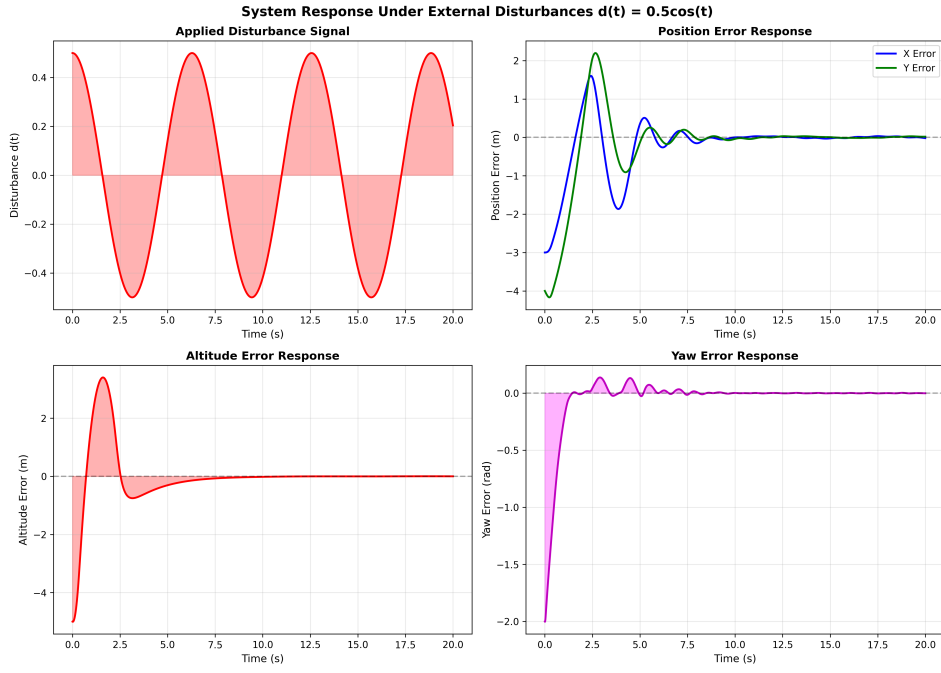


Figure 6.6: System response to random disturbances.

Expected behavior:

- slight oscillations immediately after disturbance,
- rapid recovery to the reference path,
- no large divergence or instability.

6.6 Control Effort

Control inputs (pitch, roll, yaw rate, altitude thrust/velocity) were monitored to ensure command values remained within safe limits.

Controller Effort: Total Thrust and Torques

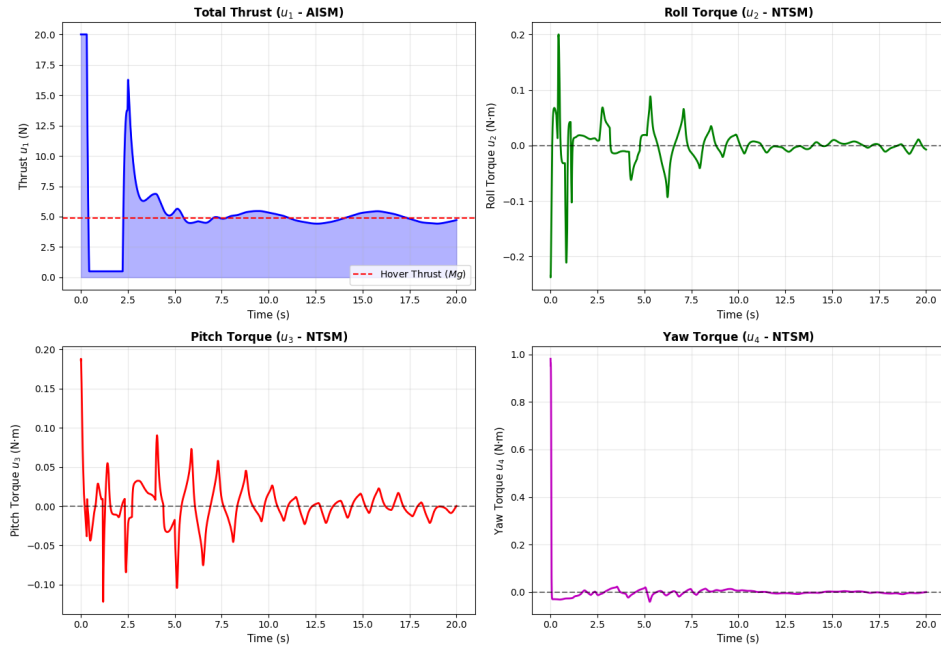


Figure 6.7: Control effort during simulation.

This analysis helps determine:

- whether the controller produces smooth outputs,

- if any inputs approach saturation,
- the feasibility of implementing the controller in real hardware.

6.7 Summary of Simulation Performance

The simulation results show that:

- the Backstepping controller provides good tracking performance for the circular x - y trajectory,
- the AISM controller maintains stable and smooth altitude tracking,
- the NTSM attitude controller and FDO ensure fast rejection of sinusoidal disturbances,
- disturbances introduce only temporary deviations, after which the system rapidly reconverges,
- overall tracking remains stable with very small steady-state errors.

These observations confirmed that the controller was ready for hardware testing.

Flight Performance Overview

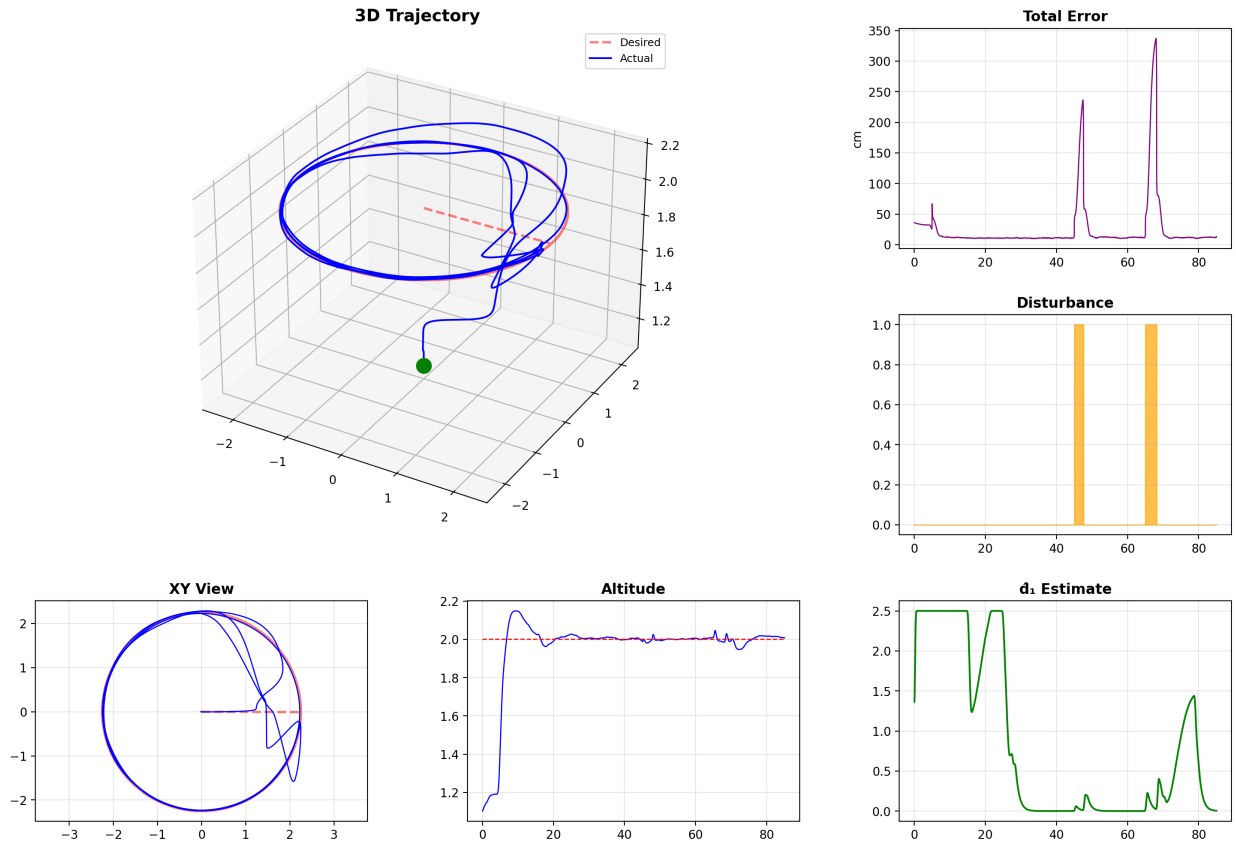


Figure 6.8: Full simulation overview including 3D trajectory, XY projection, altitude tracking, disturbance activation, total error, and disturbance estimate in a unified view.

Chapter 7

Hardware Implementation and Real-World Results

This chapter presents the hardware validation experiments performed on the Parrot AR 2.0 drone. The goal was to test whether the AISM altitude controller and the Backstepping horizontal controller operate effectively on a real quadrotor platform. The control loop frequency was kept at 50 Hz, identical to the simulation setup.

7.1 Hardware Platform

All hardware experiments were conducted on the Parrot AR 2.0 drone, controlled through ROS2 using the Python-based `pyardrone` interface. Only high-level velocity and altitude commands were available; low-level torque and moment inputs were inaccessible due to firmware restrictions.

Table 7.1: Hardware System Summary (Placeholder)

Component	Description
Drone model	Parrot AR 2.0
Control interface	<code>pyardrone</code> (Python)
Feedback sensors	IMU, altimeter
Control frequency	50 Hz
Disturbances	Manual taps, indoor airflow

7.2 Experimental Tests

Three experiments were conducted to evaluate real-world performance:

- **Hover test:** to assess altitude stability under minimal motion,
- **Circle tracking:** following the same circular trajectory used in simulations,
- **Disturbance response:** evaluating robustness under small manual taps.

7.3 Results Summary

The quadrotor successfully tracked the circular trajectory with only small overshoot. Altitude remained stable, although minor jitter appeared due to sensor noise. Manual disturbances caused brief deviations, but the controller returned the quadrotor to the desired path quickly.

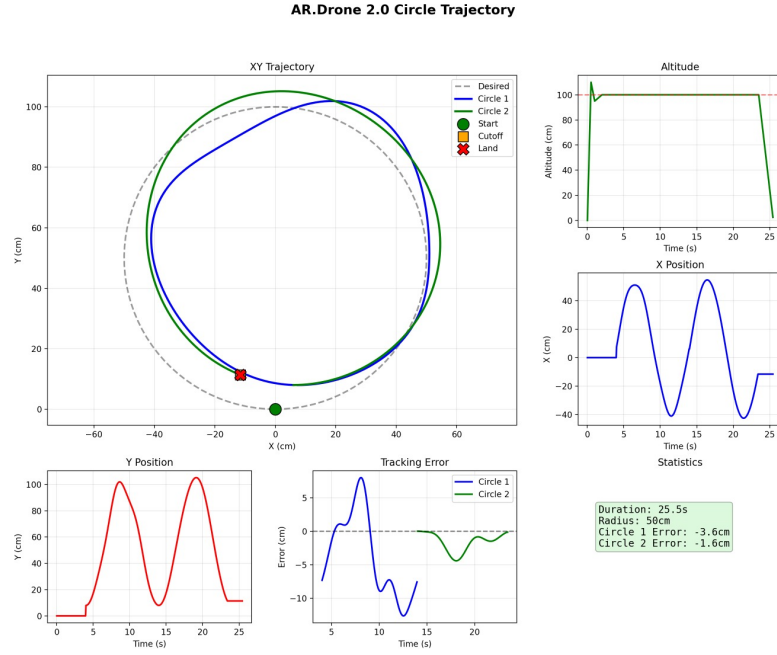


Figure 7.1: Hardware trajectory tracking.

7.4 Discussion

The main limitations encountered were:

- sensor noise causing altitude jitter,
- communication delays due to Wi-Fi transmission,
- internal firmware controlling attitude, preventing low-level tuning.

Despite these constraints, hardware behavior closely matched simulation results, demonstrating that the simplified hybrid control strategy is feasible for real-world quadrotor experiments.

Chapter 8

Conclusions, Limitations, and Future Research Directions

This project explored a simplified implementation of the Hybrid Finite-Time Control (HFTC) strategy for quadrotor trajectory tracking. The AISM altitude controller and Backstepping horizontal controller were implemented and tested in both simulation and hardware using the Parrot AR 2.0 platform with a 50 Hz control loop.

8.1 Conclusions

The main outcomes of this work are:

- successful implementation of AISM (altitude) and Backstepping (horizontal) controllers,
- accurate circular trajectory tracking in simulation and hardware,
- stable altitude behavior with limited noise-induced variations,
- fast disturbance recovery, confirming robustness of the simplified control structure.

8.2 Limitations

The primary limitations encountered during implementation include:

- absence of low-level attitude control access on the Parrot AR 2.0,
- sensor noise and communication delays affecting performance,
- inability to implement the full HFTC structure (FDO and NTSM) due to hardware restrictions.

8.3 Future Work

Potential improvements and research extensions include:

- implementing a Finite-Time Disturbance Observer (FDO) for enhanced robustness,
- integrating Nonsingular Terminal Sliding Mode (NTSM) for fast attitude tracking,
- using a PX4-based or custom-built drone with full low-level motor and torque access,
- executing the controller onboard instead of relying on offboard Wi-Fi communication.

Overall, the results demonstrate that even a reduced version of the hybrid control framework can achieve good tracking performance in both simulated and real flight scenarios.

References

- [1] N. Wang, H. Li, H. Ji, and C. Li, “Hybrid finite-time trajectory tracking control of a quadrotor,” *ISA Transactions*, vol. 91, pp. 207–220, 2019.
- [2] R. Mahony, V. Kumar, and P. Corke, “Multirotor aerial vehicles: Modeling, estimation, and control of quadrotor,” *IEEE Robotics & Automation Magazine*, vol. 19, no. 3, pp. 20–32, 2012.
- [3] Open Robotics, “ROS2 Documentation,” Available: <https://docs.ros.org>
- [4] PX4 Autopilot, “PX4 User Guide,” Available: <https://docs.px4.io>
- [5] Parrot SA, “Parrot AR.Drone Developer Guide,” Available: <https://developer.parrot.com>
- [6] J.-J. E. Slotine and W. Li, *Applied Nonlinear Control*. Prentice Hall, 1991.